
Portfolio

이 름 : 박인호

지원 분야 : [빈딕투스:디파잉 페이트]클라이언트 프로그래머



Mission System Architecture 설명서

개발 환경 : Unreal Engine 5.5.4

설계 목표

- 1 UI와 도메인 로직의 분리 ●●●●
- 2 객체 간 직접 참조 최소화 ●●●●
- 3 상태 기반(State-driven) 설계 ●●●●
- 4 확장성과 유지보수성 확보 ●●●●



전체 구조 (3계층 + Mediator 구조)

관계 원칙

- 1 Widget은 Mission을 모름 ○○○○
- 2 Mission은 Widget을 모름 ○○○○
- 3 MissionManager만 둘 다 참조 ○○○○
- 4 → 결합도 감소 ○○○○

```
01. UI Layer
   - UMainMissionWidget.

02. Control Layer
   - UMissionManager(Mediator)

03. Domain Layer
   - UBaseMission(CurrentMission)
```

① 명령 흐름 (Command Flow)

사용자 입력이 아래로 내려가는 흐름



1. 버튼 클릭 발생
2. Widget이 MissionManager의 함수 호출
3. MissionManager가 CurrentMission의 로직 함수 호출
 - AgreeMission()
 - Disagree()
 - SuccessMission()
 - FailedMission()

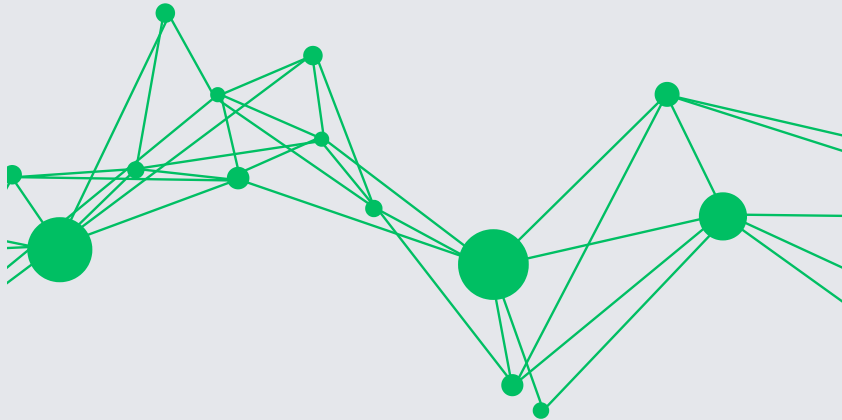
② 이벤트 흐름 (Event / Notify Flow)

상태 변경이 위로 전달되는 흐름

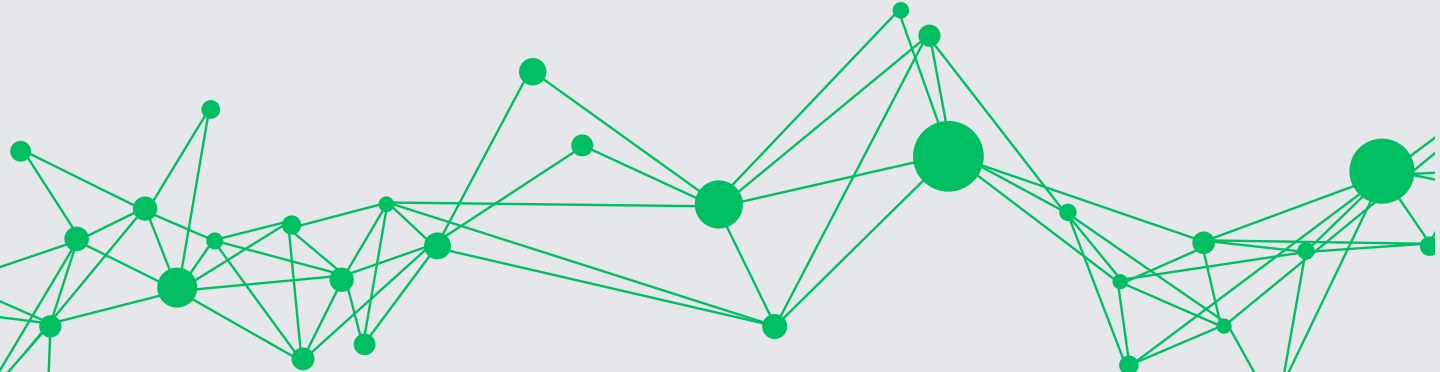


1. BaseMission 내부에서 SetState() 호출
2. OnMissionStateChanged.Broadcast()
3. MissionManager가 수신
4. Widget에 새로운 상태 전달
5. UI 외형 변경

핵심 구조 정리



객체	역할
UMainMissionWidget	입력 및 표현
UMissionManager	중재자 (Mediator)
UBaseMission	상태 및 로직 소유



사용된 디자인 패턴

① Mediator Pattern

- MissionManager가 모든 통신을 중재
- Widget ↔ Mission 직접 통신 없음
- 모든 흐름은 Manager를 통해 전달

② Observer Pattern

- MissionState 변경 시 Delegate 사용
- Manager가 상태 변경 감지
- UI 갱신

① 상태 기반 처리 구조

상태	Left Button	Right Button
Ready	AgreeMission	Disagree
InProgress	SuccessMission	FailedMission

② 객체 생명주기 관리

ClearCurrentMission() 수행 시:

- Delegate 해제
→ Dangling Delegate 방지
- CurrentMission nullptr 처리
- Widget 숨김

설계 장점



• 1

UI와 로직 완전 분리



• 2

결합도 감소



• 3

확장 용이 (다중 미션 가능)



• 4

유지보수성 향상



• 5

멀티플레이 확장 가능 구조





Contact

마무리

감사합니다

- hellop1013@naver.com
- 010-6782-1013

Thank you for reviewing my portfolio.

